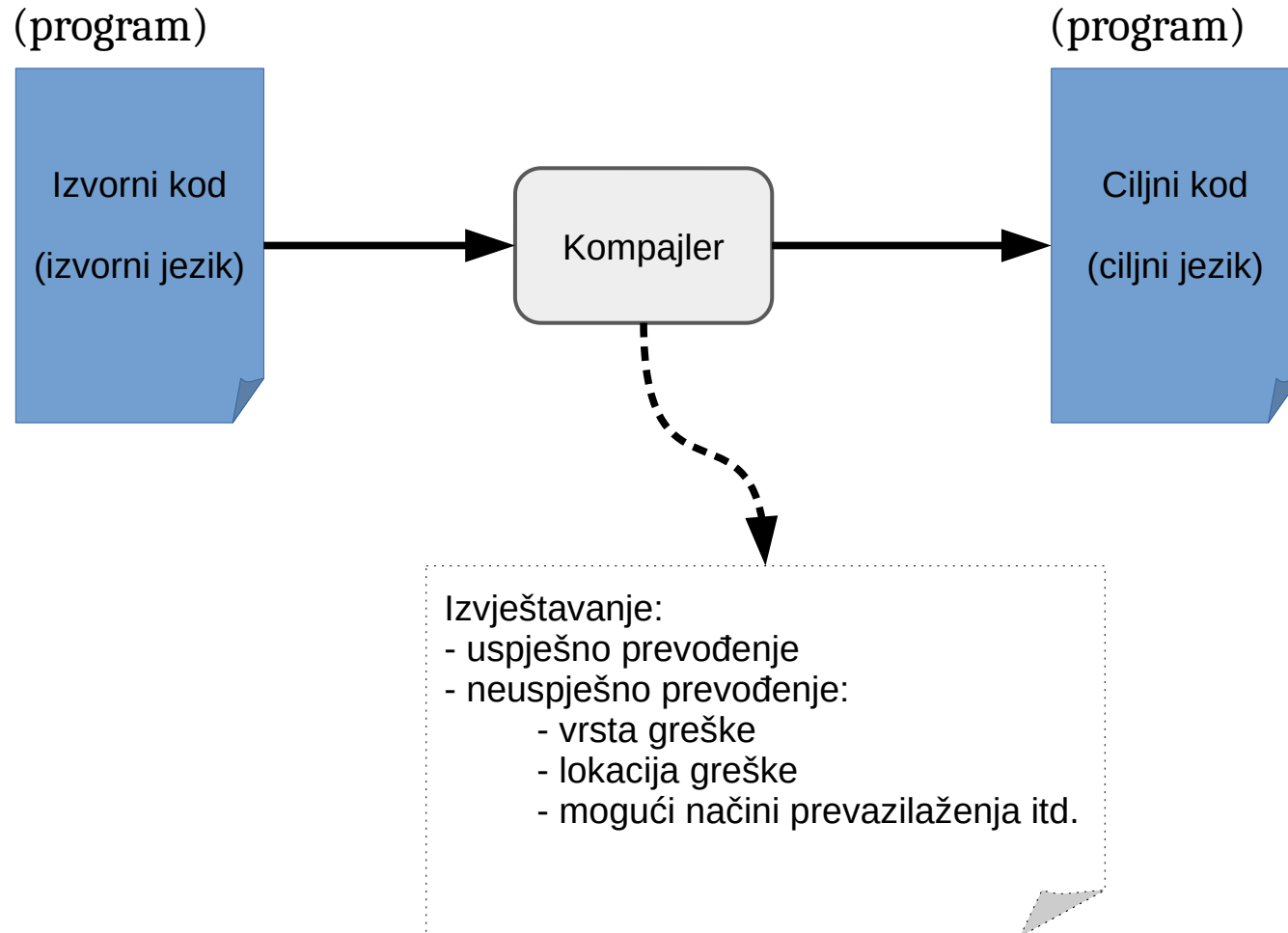


RI402

Dizajn kompajlera

Edin Pjanić

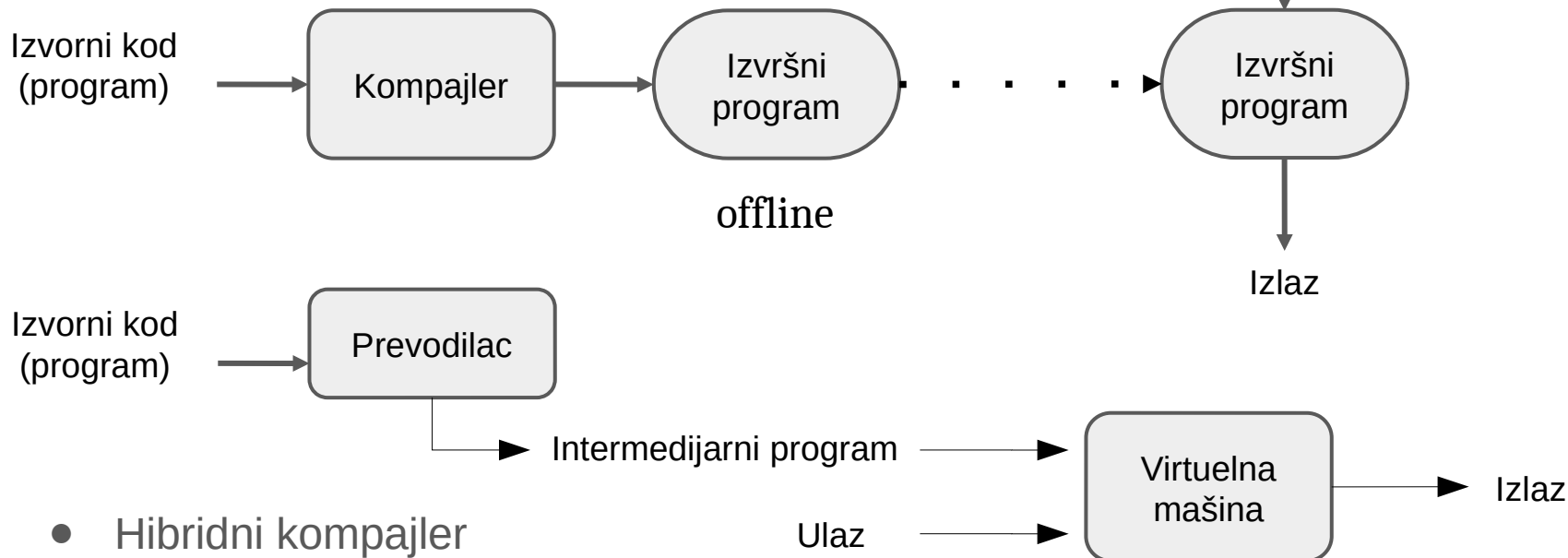
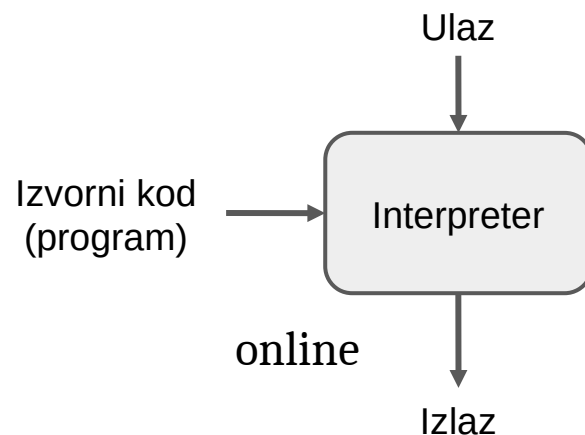
Kompajler: Procesor jezika



Pristupi

Dva glavna pristupa implementaciji programskih jezika:

- Kompajler
- Interpreter



- Hibridni kompajler

Početak

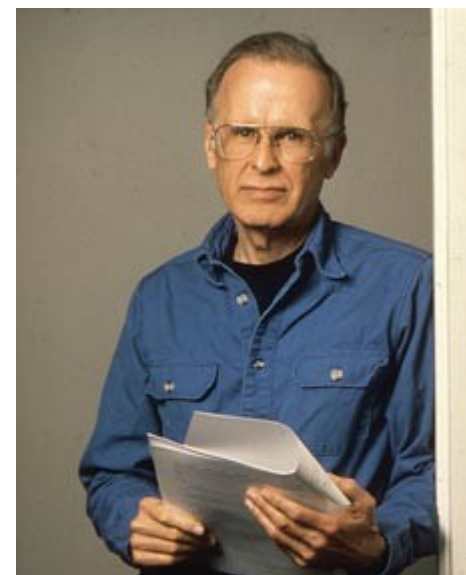
1950-te godine

- Programiranje u assembleru
- Cijena hardvera viša nego ikad
- Cijena razvoja softvera velika



1953. John Backus - Speedcoding

- Poseban dio softvera
- Mogućnost korištenja instrukcija “višeg nivoa” sa FP brojevima
 - sin, cos, exp, ...
- Izvršava se kao interpreter
 - Brži razvoj softvera
 - 10-20 x sporije izvršavanje programa
 - Interpreter zauzima 30% memorije (300 bytes)



1924.-2007.

Speedcoding za IBM 701

OP ₁ Operations				
Alphabetical Code	Numerical Code	Name	Description	
ADD	658	Add	$Q(A) + Q(B) = Q(C)$	
SUB	696	Subtract	$Q(A) - Q(B) = Q(C)$	
ADDAB	699	Add absolute	$Q(A) + Q(B) = Q(C)$	
ABADD	703	Absolute add	$ Q(A) + Q(B) = Q(C)$	
SUBAB	707	Subtract absolute	$Q(A) - Q(B) = Q(C)$	
ABSUB	711	Absolute subtract	$ Q(A) - Q(B) = Q(C)$	
MPY	715	Multiply	$[Q(A)] \times [Q(B)] = Q(C)$	
NGMPY	731	Negative multiply	$-[Q(A)] \times [Q(B)] = Q(C)$	
DIV	734	Divide	$[Q(A)] \div [Q(B)] = Q(C)$	
NGDIV	748	Negative divide	$-[Q(A)] \div [Q(B)] = Q(C)$	
SQRT	782	Square root	$\sqrt{Q(A)} = Q(C)$	
SINE	780	Sine	$\sin [Q(A)] = Q(C)$	
ARTAN	781	Arc tangent	$\tan^{-1} [Q(A)] = Q(C)$	
EXP	783	Exponential	$e^{[Q(A)]} = Q(C)$	
LN	784	Logarithm	$\log_e [Q(A)] = Q(C)$	
MOVE	690	Move	The block of information stored in electrostatic cells A to B is stored in electrostatic cells C to $C + B - A$. See Appendix E.	

Bušene kartice za speedcoding

PROGRAM LABEL	LOCATION	ALPHA-BETIC OP ₁	0000	A ADDRESS	B ADDRESS	C ADDRESS	ALPHA-BETIC OP ₂	D ADDRESS	0000	NUMERIC OP ₁	NUMERIC OP ₂	REMARKS
00000000	0000	000000	0000	0000	0000	0000	000000	0000	0000	0000	0000	00000000000000000000000000000000
11111111	1111	111111	1111	1111	1111	1111	111111	1111	1111	1111	1111	11111111111111111111111111111111
22222222	2222	222222	2222	2222	2222	2222	222222	2222	2222	2222	2222	22222222222222222222222222222222
33333333	3333	333333	3333	3333	3333	3333	333333	3333	3333	3333	3333	33333333333333333333333333333333
44444444	4444	444444	4444	4444	4444	4444	444444	4444	4444	4444	4444	44444444444444444444444444444444
55555555	5555	555555	5555	5555	5555	5555	555555	5555	5555	5555	5555	55555555555555555555555555555555
66666666	6666	666666	6666	6666	6666	6666	666666	6666	6666	6666	6666	66666666666666666666666666666666
77777777	7777	777777	7777	7777	7777	7777	777777	7777	7777	7777	7777	77777777777777777777777777777777
88888888	8888	888888	8888	8888	8888	8888	888888	8888	8888	8888	8888	88888888888888888888888888888888
99999999	9999	999999	9999	9999	9999	9999	999999	9999	9999	9999	9999	99999999999999999999999999999999

IBM FOR SPEED CODING SYSTEM INSTRUCTION CARD

IBM 030521

FORTRAN I

1954. - 1957. John Backus - FORTRAN (FORmula TRANslation)

- "Speedcoding": problem je što se formule interpretiraju
- Ideja: prevesti kod višeg nivoa u asemblerske instrukcije prije izvršavanja programa

1958.

- 50 % softvera je u FORTRAN-u
- Vrijeme razvoja softvera upolovljeno
- Prvi kompajler
- Veliki uticaj na razvoj računarskih znanosti
- Struktura kompajlera zadržana do danas

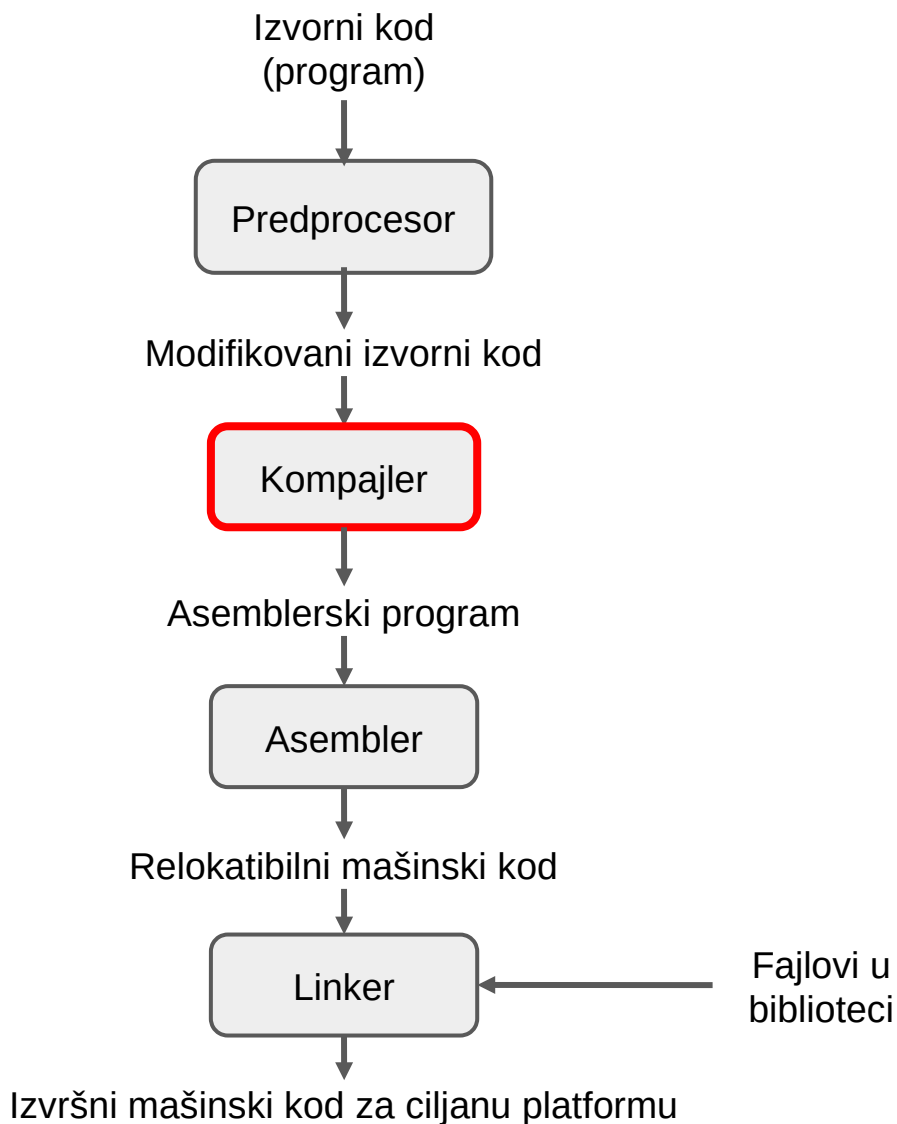
```

C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
C INPUT - TAPE READER UNIT 5, INTEGER INPUT
C OUTPUT - LINE PRINTER UNIT 6, REAL OUTPUT
C INPUT ERROR DISPLAY ERROR OUTPUT CODE 1 IN JOB CONTROL LISTING
  READ INPUT TAPE 5, 501, IA, IB, IC
  501 FORMAT (3I5)
C IA, IB, AND IC MAY NOT BE NEGATIVE OR ZERO
C FURTHERMORE, THE SUM OF TWO SIDES OF A TRIANGLE
C MUST BE GREATER THAN THE THIRD SIDE, SO WE CHECK FOR THAT, TOO
  IF (IA) 777, 777, 701
701 IF (IB) 777, 777, 702
702 IF (IC) 777, 777, 703
703 IF (IA+IB-IC) 777, 777, 704
704 IF (IA+IC-IB) 777, 777, 705
705 IF (IB+IC-IA) 777, 777, 799
777 STOP 1
C USING HERON'S FORMULA WE CALCULATE THE
C AREA OF THE TRIANGLE
799 S = FLOATF (IA + IB + IC) / 2.0
  AREA = SQRTF( S * (S - FLOATF(IA)) * (S - FLOATF(IB)) *
+ (S - FLOATF(IC)))
  WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA
601 FORMAT (4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
+ 13H SQUARE UNITS)
  STOP
  END

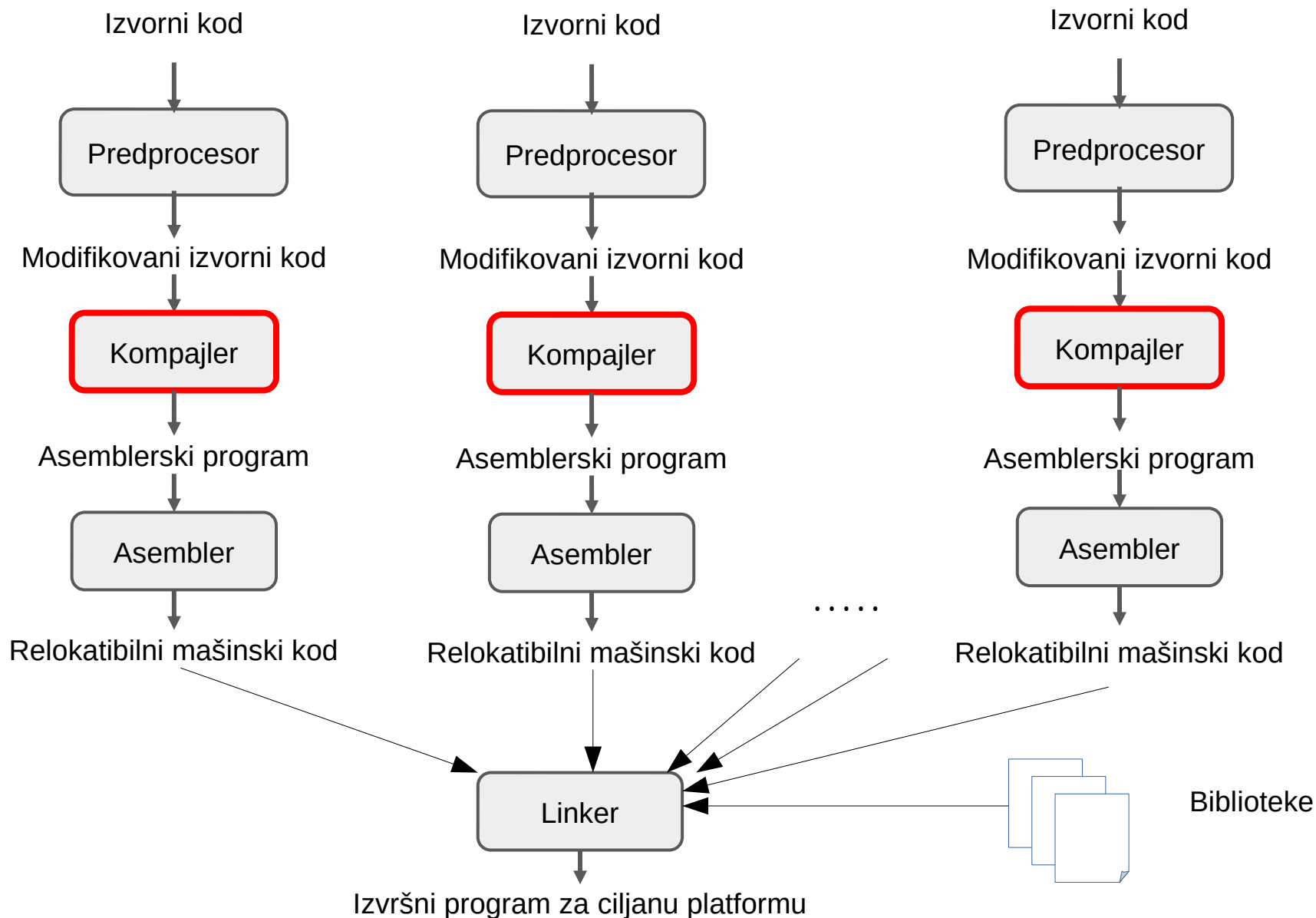
```

Sistem za procesiranje jezika (programskog)

Osim kompajlera, često su potrebni i dodatni programi.



Proizvodnja izvršnog programa



Struktura kompajlera

1. Leksička analiza (skeniranje) - lekser
2. Sintaksna analiza (rašćlanjivanje) - parser
3. Semantička analiza
4. Optimizacija
5. Generisanje koda

Faze 1-3: **front end** - primarno zadužen za razumijevanje izvornog jezika

Faze 4-5: **back end** - primarno zadužen za poznavanje ciljne mašinske arhitekture.

Leksička analiza

Analogija sa govornim jezikom: prepoznati riječi (najmanji nivo iznad slova/znaka).

Ovo uključuje prepoznavanje riječi, razmaka, znakova interpunkcije, kao i vrste riječi.

Ovo je rečenica.

Međutim, ovaj proces nije trivijalan. Razmotrimo sljedeći niz znakova:

Dali jeo vo reče nica.

Leksička analiza

U kontekstu programskih jezika, leksički analizator dijeli izvorni tekst (program) u riječi koje zovemo "tokeni":



if **x==y** **then** **z=1;** **else** **z=2;**

Prepoznate logičke jedinice teksta:

<IF> <razmak> <id[x]>

<operator[poređenje_jednakosti]> <id[y]> <razmak>

<THEN> <razmak> <id[z]> <operator[dodjeljivanje]>

<konstanta[1]> <kraj-naredbe> <razmak> <ELSE>

<razmak> <id[z]> <operator[dodjeljivanje]>

<konstanta[2]> <kraj-naredbe>

Sintaksna analiza (parsiranje)

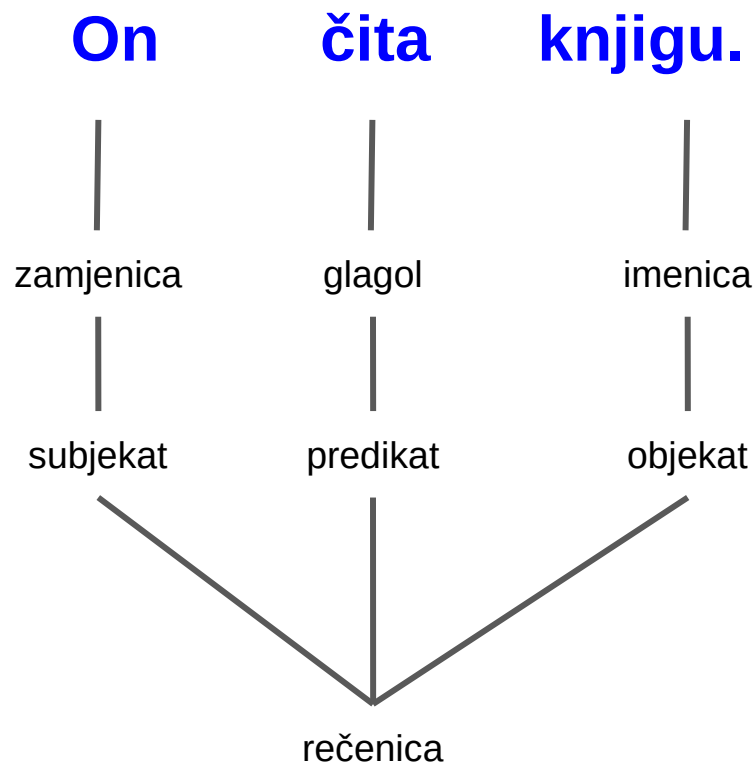
Nakon prepoznavanja riječi slijedi prepoznavanje rečenice i prepoznavanje njene strukture, odnosno ispravnosti te strukture (sintakse).

Mogli bismo reći da je parsiranje u stvari skiciranje rečenice.

Razultat skiciranja je **stablo**.

Skiciranje rečenice

Primjer jednostavne rečenice iz govornog jezika:

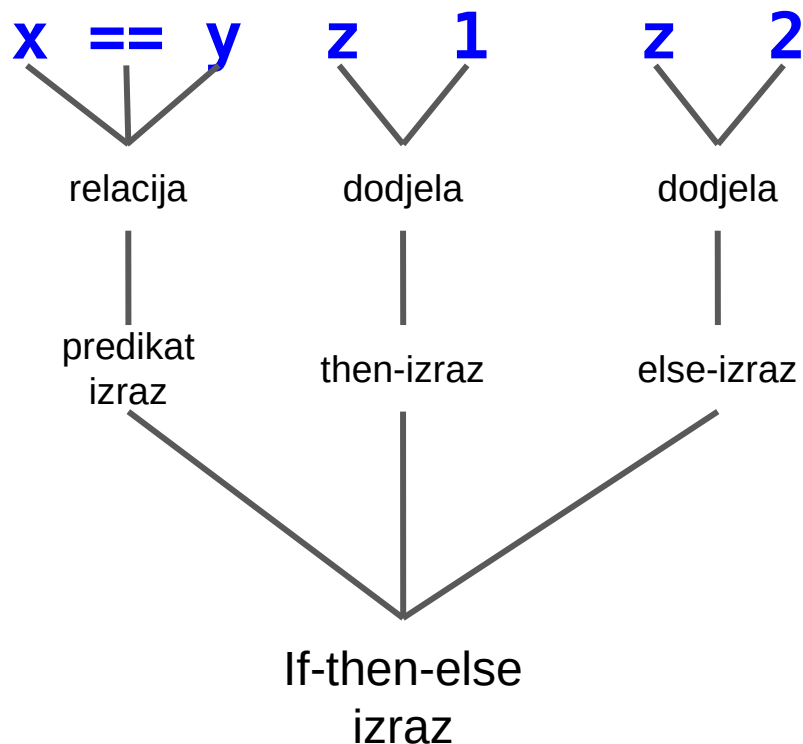


Parsiranje programa

Parsiranje programskog koda se vrši na sličan način. Razmotrimo sljedeće:

if x==y then z=1; else z=2;

Dobijamo stablo:



Semantička analiza

- Nakon prepoznavanja strukture rečenice, slijedi razumijevanje njenog značenja.
 - Međutim, razumijevanje značenja je za kompajlere preteško.
- Kompajleri uglavnom vrše izvjesnu analizu kako bi prepoznali razne nekonzistentnosti u programu.

Semantička analiza govornog jezika

Razmislimo o rečenici:

Suljo i Mujo su morali čekati Fatu, pa su zakasnili na njihovu zabavu.

Ili gore:

Mujo kaže da je Mujo morao čekati Fatu, pa su zakasnili na njihovu zabavu.

Najgore:

Jeste li gledali emisiju “Život na vazi”?

Možda je pisac mislio na ovu emisiju:



Semantička analiza programskog jezika

Programski jezici imaju stroga pravila kako bi se otklonila nedvosmislenost.
Poslatrajmo C++ kod:

```
{  
    string a = "Mujo";  
    {  
        string a = "Suljo";  
        cout << a;  
    }  
}
```

Na standardni izlaz će se ispisati riječ "Suljo".

Osim opsega, kompajleri vrše i druge semantičke provjere, kao npr. provjera tipova, redoslijed izvršavanja ili grupisanja izraza itd.

Optimizacija

U kontekstu programskog jezika, cilj optimizacije je ušteda nekog resursa:

- Brže izvršavanje programa (ušteda vremena)
- Manje korišćenje memorije
- Manje korišćenje energije potrebne za izvršavanje programa
- Ušteda nekog drugog resursa...

Generisanje koda

Rezultat faze generisanja koda:

- Asemblerski kod
- Mašinski kod
- Kod u nekom drugom programskom jeziku

Intermedijarni jezici

- Jezik u nekoj fazi prevođenja programa
 - osim u prvoj i zadnjoj fazi
- Mnogi kompajleri u pojedinim fazama vrše prevođenje programa iz jednog intermedijarnog oblika u drugi
- Olakšava se optimizacija i generisanje koda

Faze kompajlera - konačno

